VU Amsterdam Coding Café - Bytes & Bites

4 November 2025, Amsterdam

How to make your research software ready for publishing?

Dr. Serkan Girgin [1, 2]

s.girgin@utwente.nl

https://linkedin.com/in/serkan-girgin/

Faculty of Geo-information Science and Earth Observation

HoD, Centre of Expertise in Big Geodata Science [1]

Associate Professor, Department of Geo-information Processing [2]

UNIVERSITY OF TWENTE. | ITC

Publishing plays a critical role in the **development**, **recognition**, and **sustainability** of research software, particularly as software becomes a central output of scientific work.

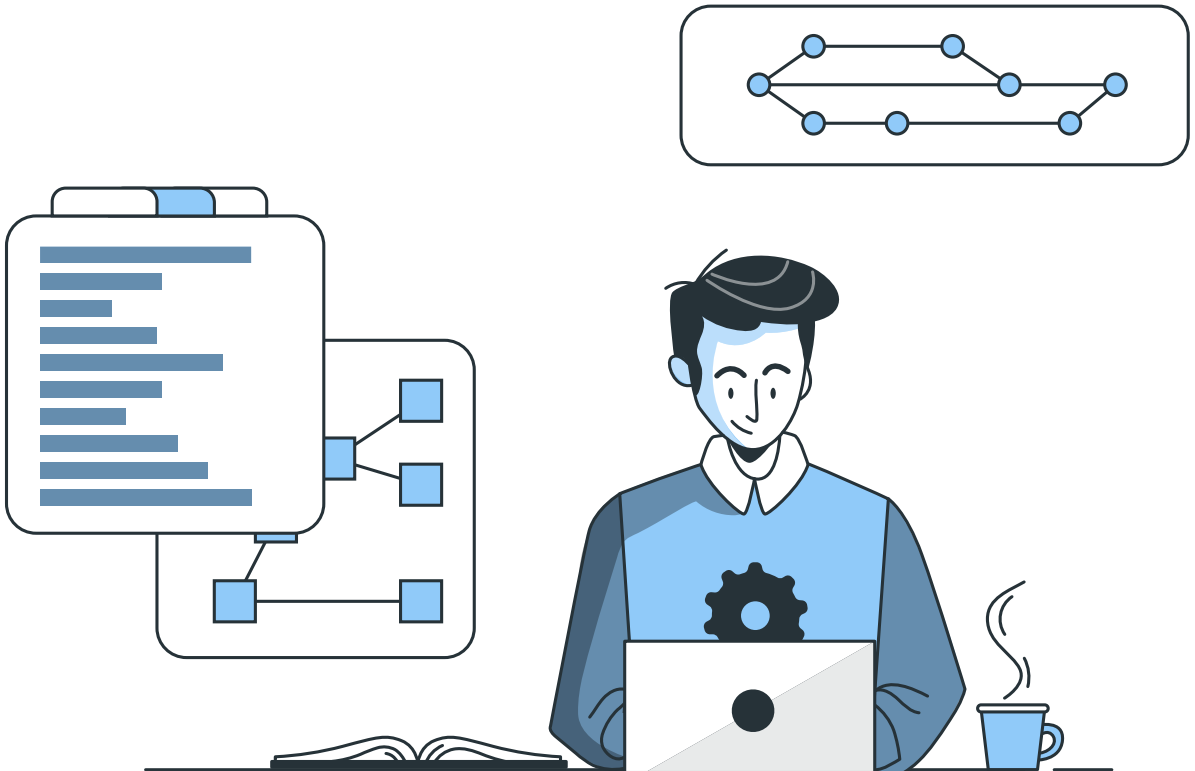


Illustration by Storyset.com

Following the **best practices** for research software development is fundamental for a **modern, open, and sustainable** research software.

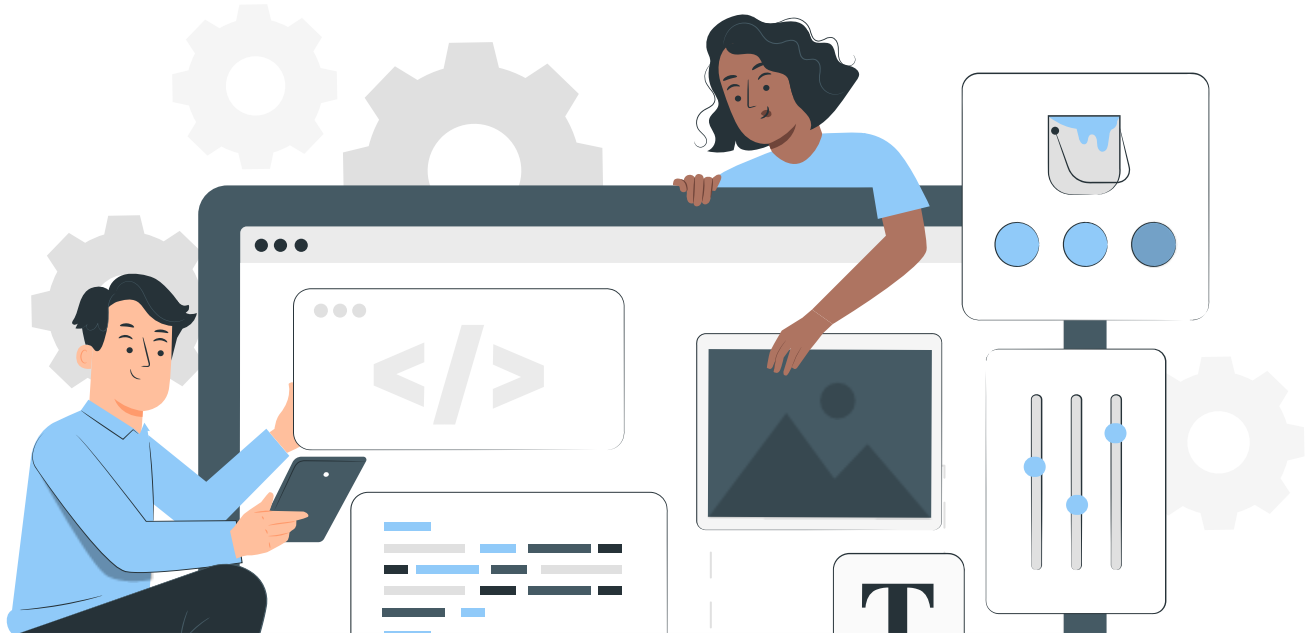


Illustration by Storyset.com

They also facilitate integration and reduce repetitive efforts through **interoperability and reuse**

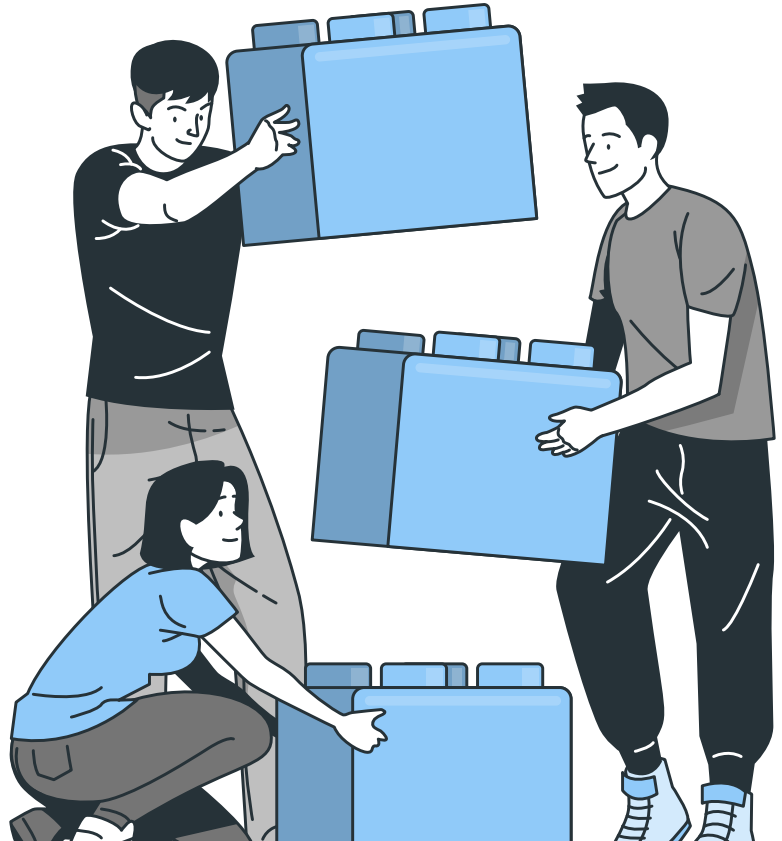


Illustration by Storyset.com

However, **implementing** rapidly changing* research software development best practices **is challenging** for research software developers*.

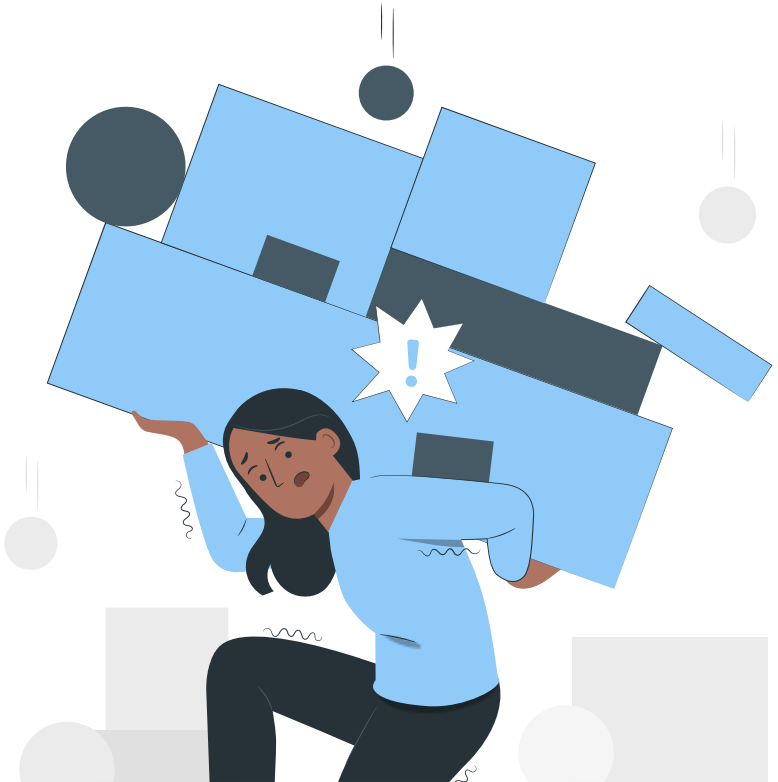


Illustration by Storyset.com

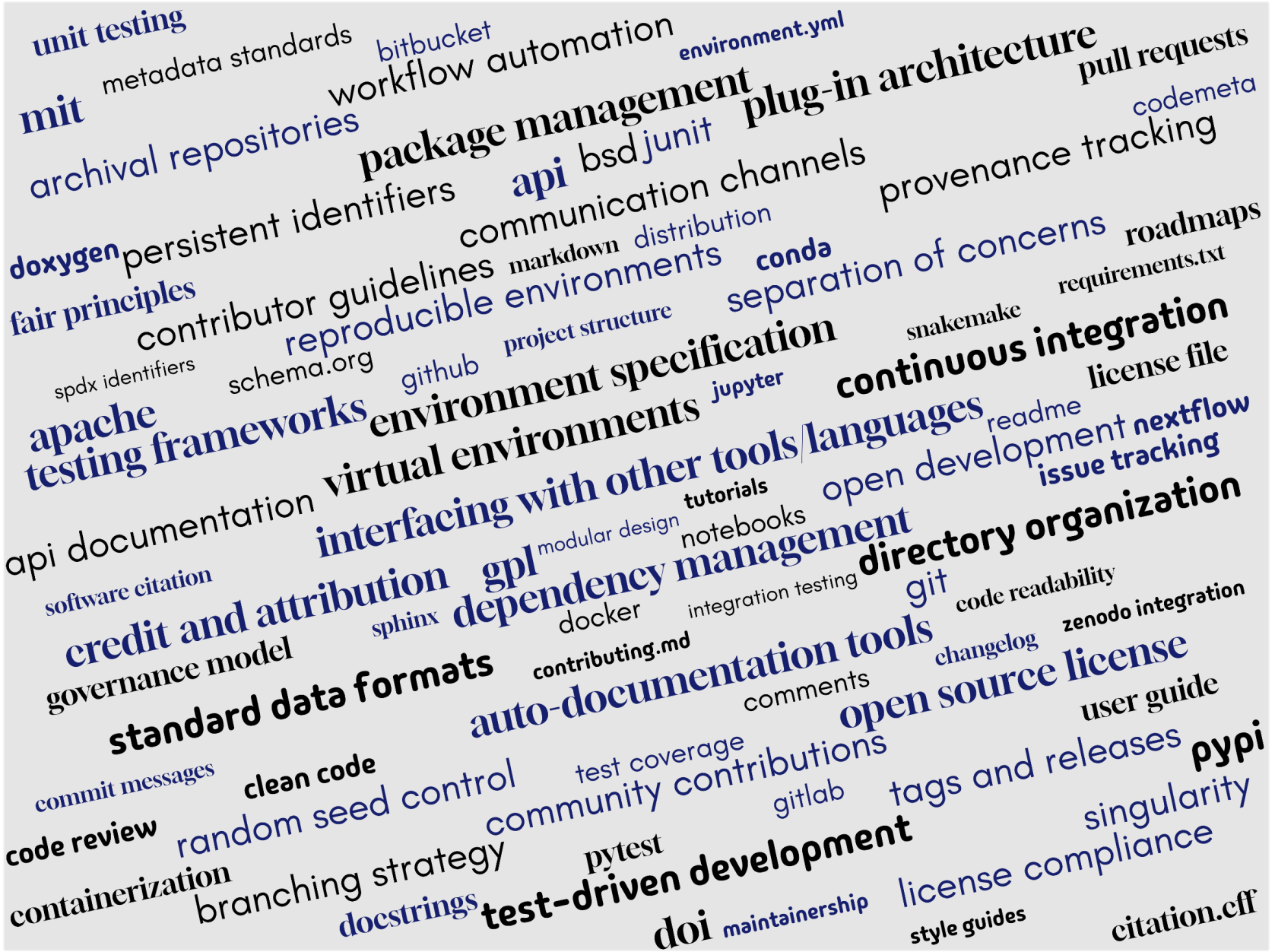
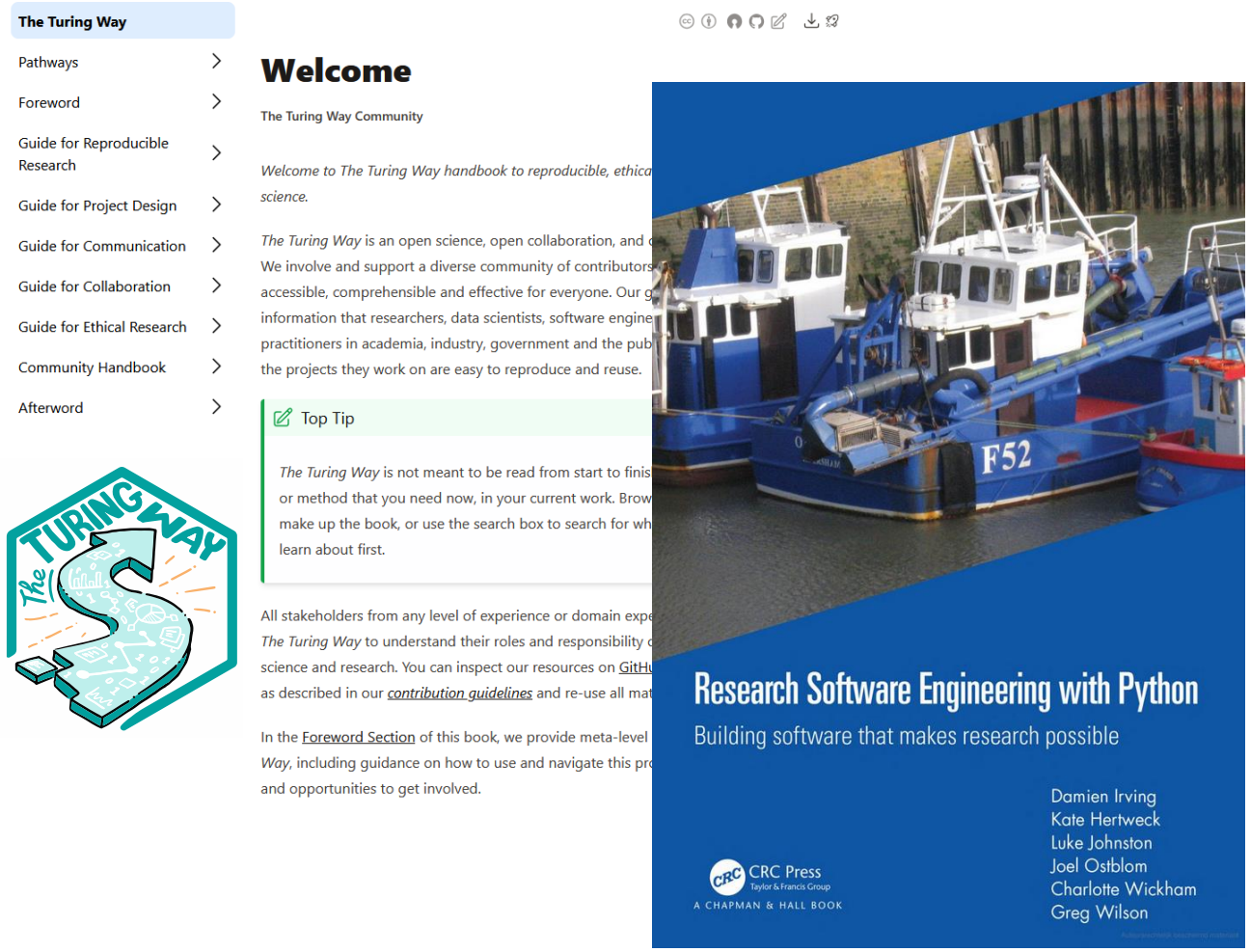


Illustration by WordClouds.com



One can **master** the best practices with dedication and curiosity

One can get support from an **AI agent** to follow the best practices

I have developed a research software for my MSc thesis. Can you check it and make sure that all best practices in research software development are followed, including license, citation, code documentation, user documentation, etc?

Absolutely — I can help you review your research software for compliance with research software development best practices. Please upload the full software package (source code and related files such as README, LICENSE, documentation, etc.), or share a link to its repository (e.g., GitHub, GitLab, or Bitbucket).

Illustration by Storyset.com

Once I have access, I will evaluate the software based on key best practices, including:

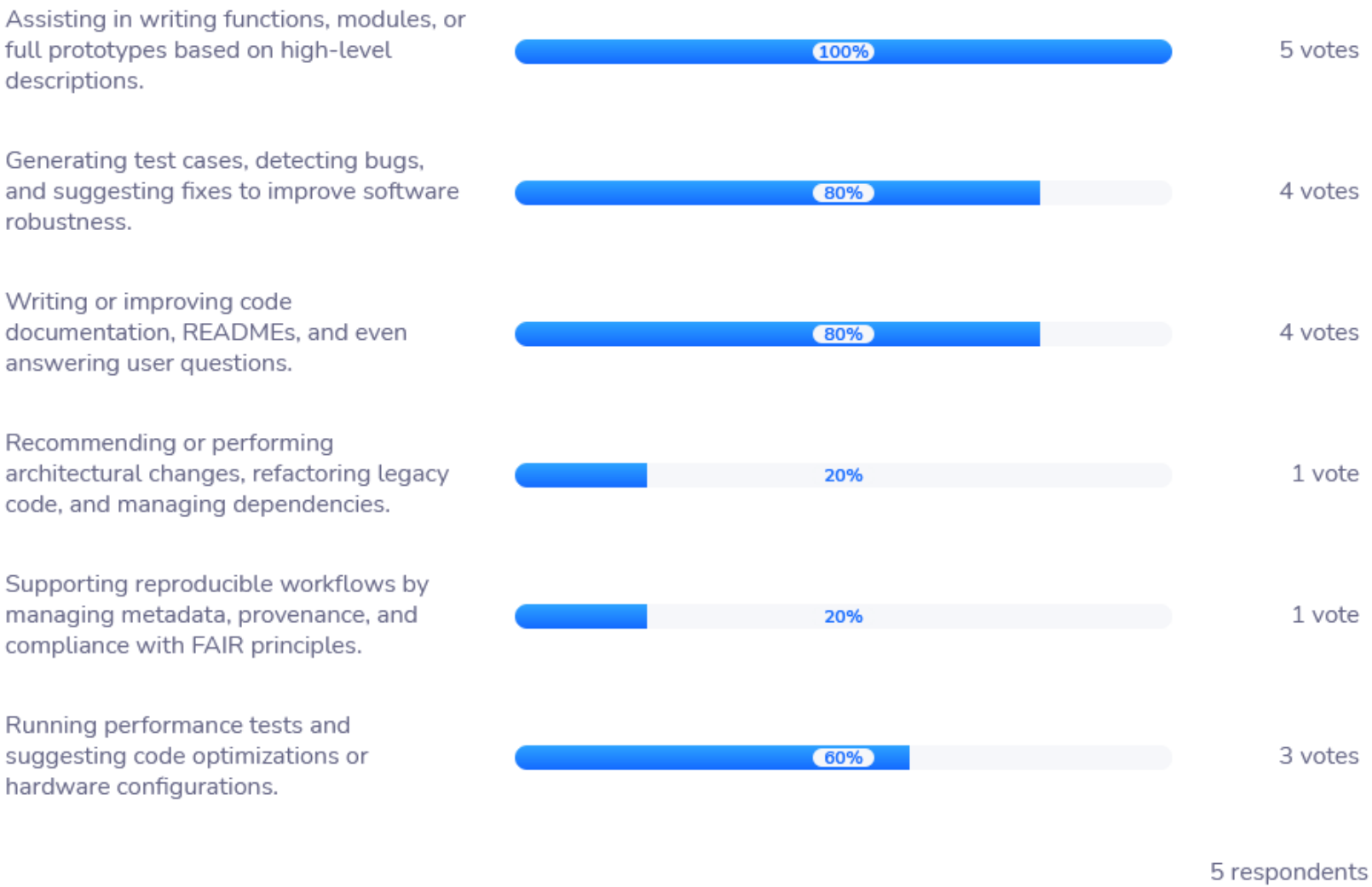
1. **Licensing**
   - Open-source license file (e.g., MIT, GPL, BSD) included?
   - License clearly stated in the README?
2. **Citation and Credit**
   - `CITATION.cff`, `codemeta.json`, or a citation section in the README?
   - Authors and contributors acknowledged?
3. **Code Documentation**
   - Docstrings in functions, classes, and modules?
   - Clear, consistent, and minimal examples provided inline?
4. **User Documentation**
   - Installation instructions?
   - Usage examples and CLI/API documentation?
   - Configuration options and dependencies listed?
5. **Repository Structure**
   - Logical and consistent folder structure?
   - Clear naming conventions?
6. **Testing and Reproducibility**
   - Unit and/or integration tests included?
   - Example datasets or synthetic data for testing?
   - Reproducibility instructions (e.g., `requirements.txt`, `environment.yml`, `Dockerfile`)?
7. **Version Control and Releases**
   - Git version control used properly (commits, tags)?
   - Versioning (e.g., SemVer) applied?
8. **Continuous Integration/Deployment (CI/CD)**
   - Automated testing setup (e.g., GitHub Actions, GitLab CI)?
   - Build/test badges in the README?

Once you upload the code or share the repository, I'll give you:

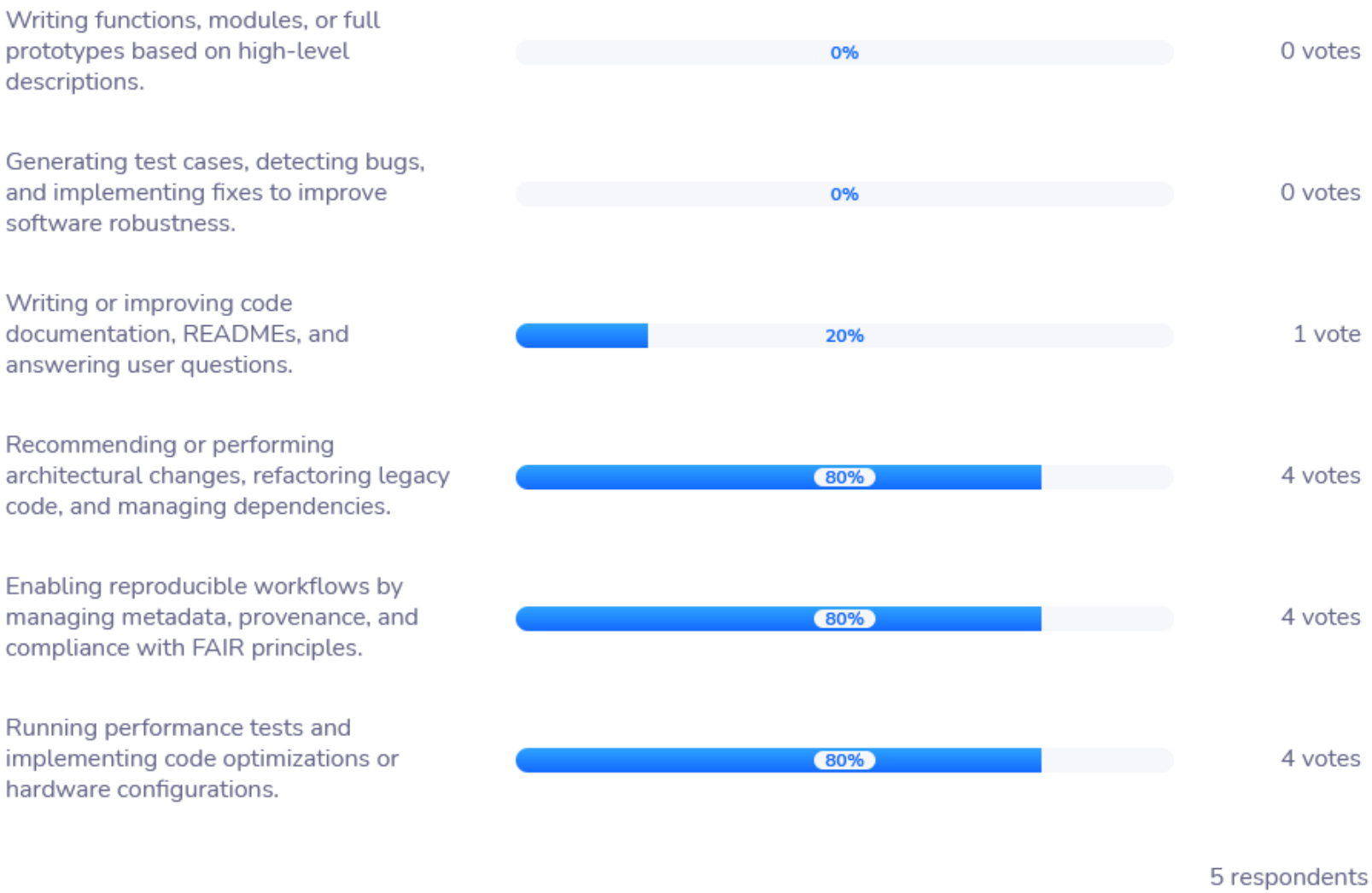
- A **detailed assessment report**.
- **Recommendations** for anything that's missing or could be improved.
- Optional help in writing or improving docs, license files, and more.

Go ahead and share the code or link when ready.

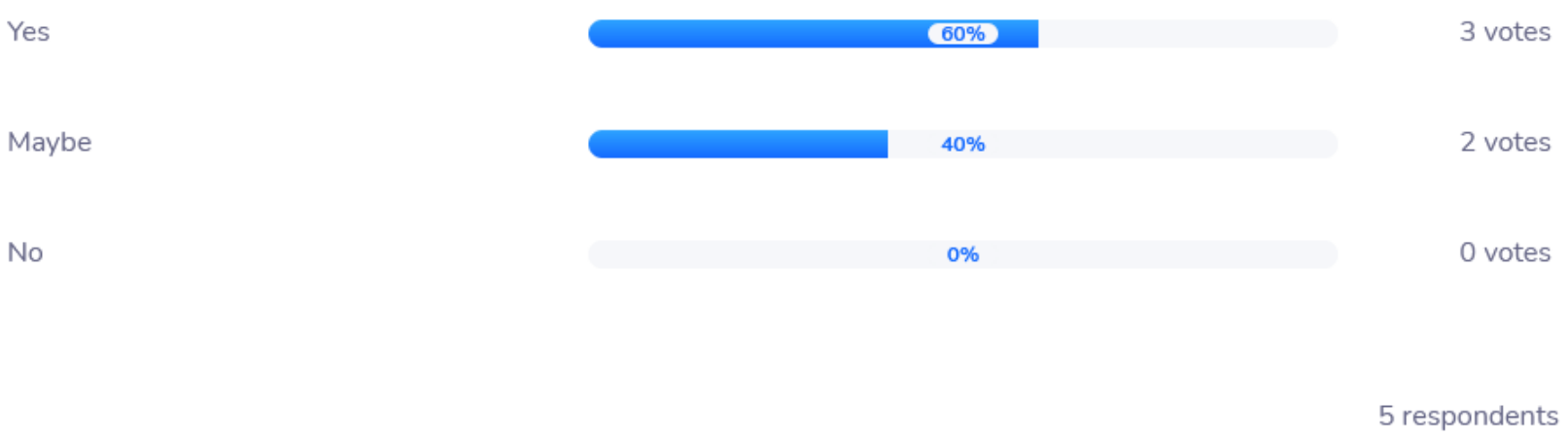
What could be the role of AI agents in developing research software?



What could be our (humans) role in developing research software?



Should we (humans) continue to develop research software?



Modern **research software templates** to initiate research software projects and **tools to check conformity** with best practices throughout the development can significantly lower the barriers.

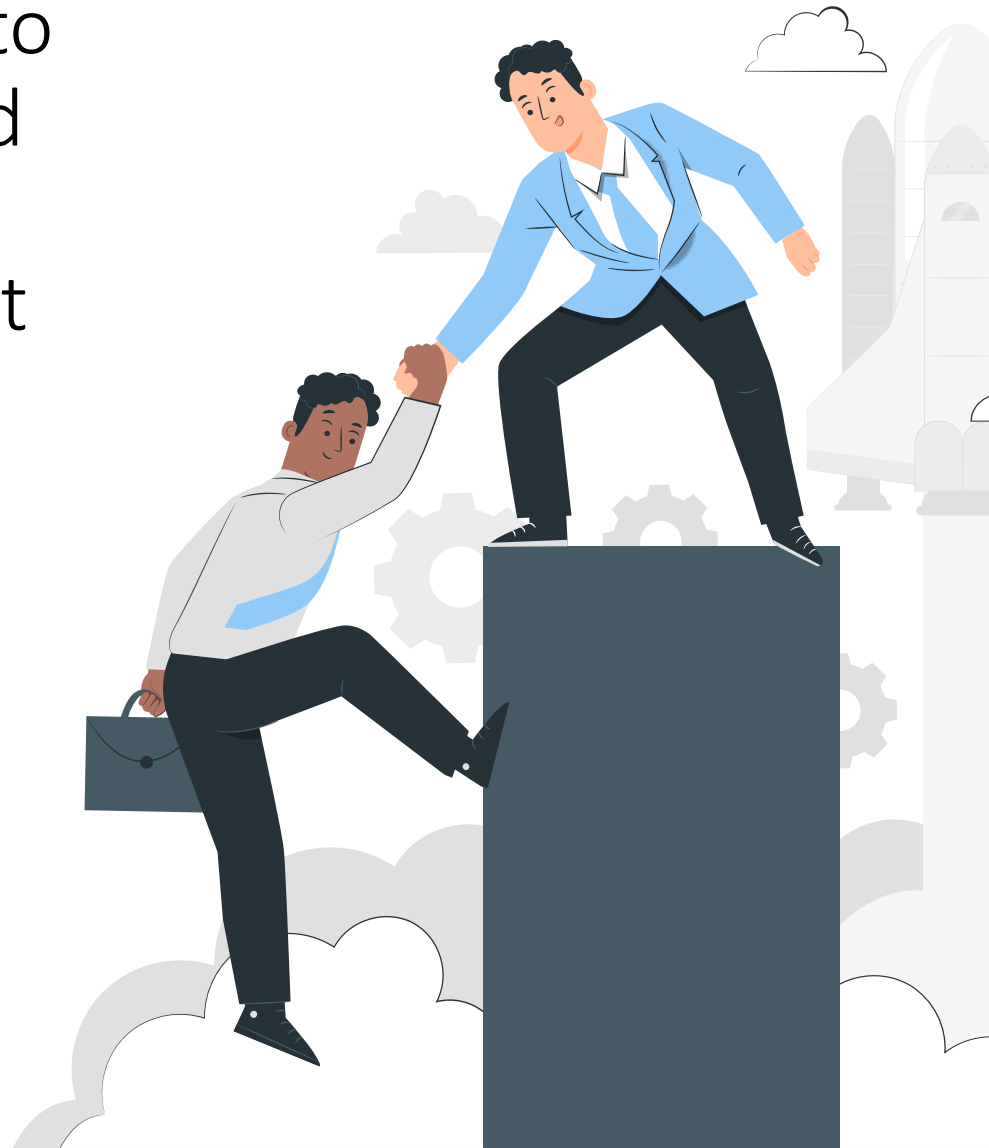


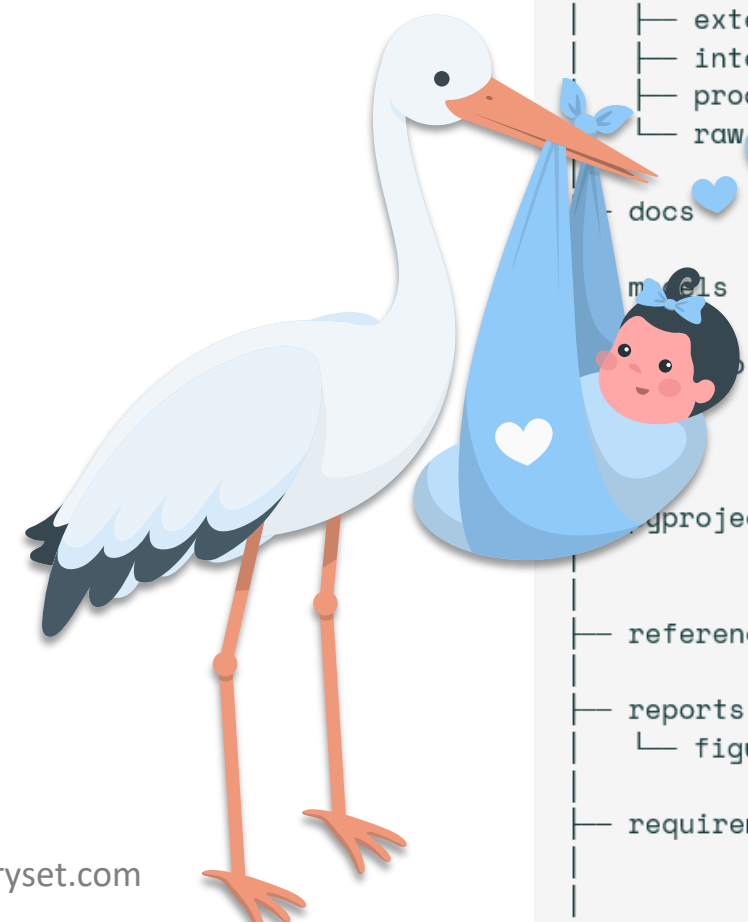
Illustration by Storyset.com

They can also help to **reduce errors**, **enhance software longevity**, and **facilitate collaboration** by automating the use of standards and promoting maintainable code

Illustration by Storyset.com

A research software template provides the **structure and boilerplate code** for a research software project*

- Code and resource layout
- Version control
- License
- Citation
- Documentation
- Code documentation
- Packaging
- Unit testing
- Publishing
- …



```
├── LICENSE            <- Open-source license if
├── Makefile           <- Makefile with convenier
├── README.md          <- The top-level README fo
├── data
│   ├── external       <- Data from third party s
│   ├── interim        <- Intermediate data that
│   ├── processed      <- The final, canonical da
│   └── raw            <- The original, immutable
│
├── docs               <- A default mkdocs projec
│
├── models             <- Trained and serialized
│
├── notebooks          <- Jupyter notebooks. Nami
│                         the creator's initials
│                         `1.0-jqp-initial-data-e
│
├── pyproject.toml     <- Project configuration f
│                         {{ cookiecutter.module_
│
├── references         <- Data dictionaries, manu
│
├── reports            <- Generated analysis as H
│   └── figures        <- Generated graphics and
│
├── requirements.txt   <- The requirements file f
│                         generated with `pip fre
│
```

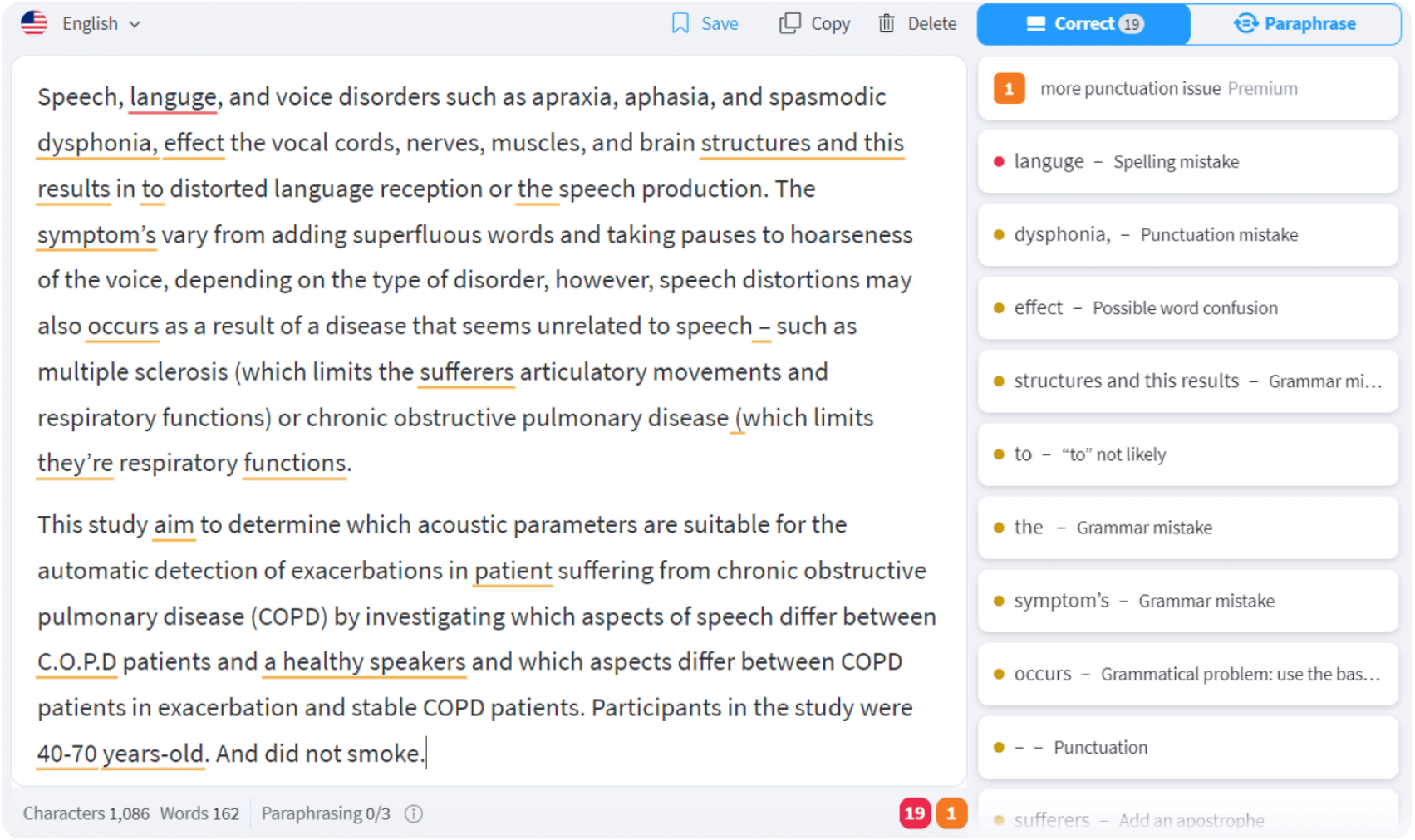
Illustration by Storyset.com

A research software conformity check tool provides insights about the **structure and content** of a research software project*

- Code and resource layout
- Version control
- License
- Citation
- Documentation
- Code documentation
- Packaging
- Unit testing
- Publishing
- …

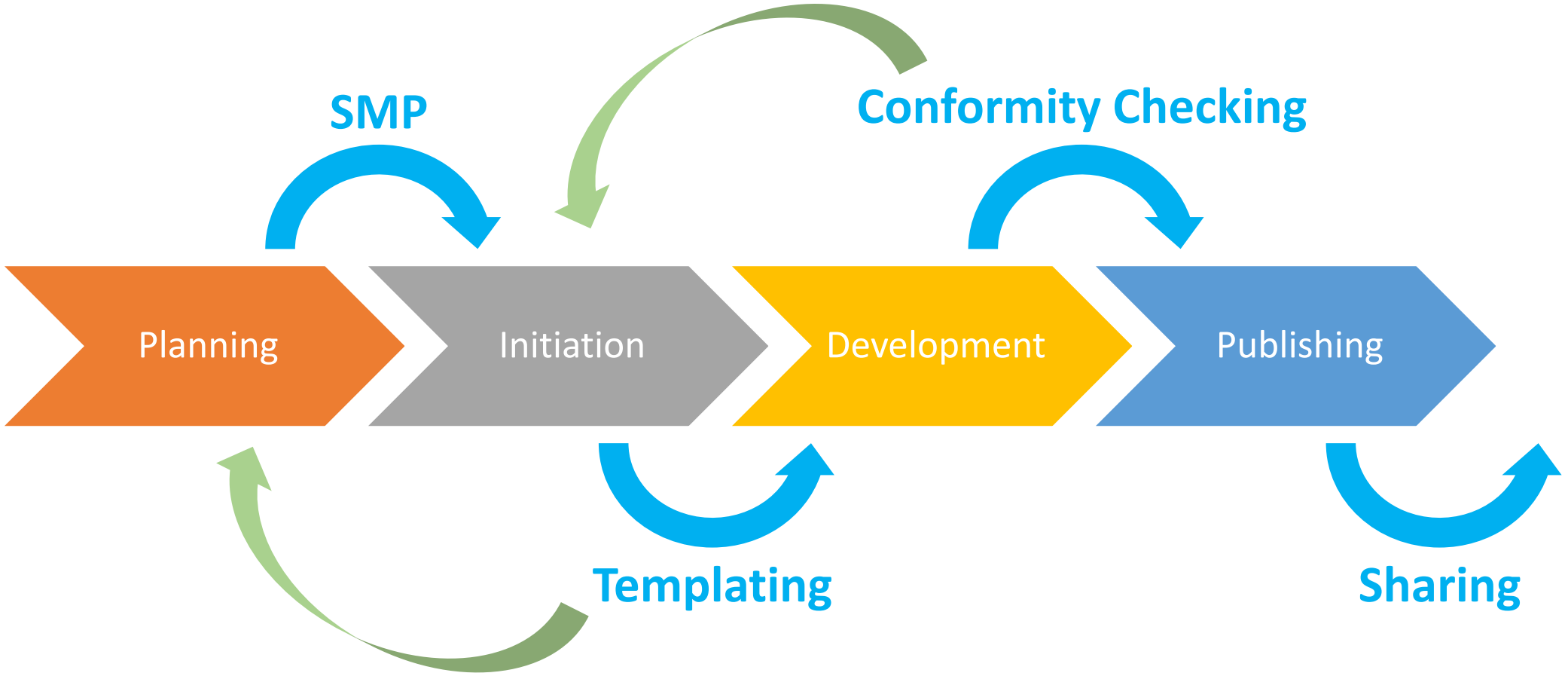
Illustration by Storyset.com

We use **checking tools** quite often in our professional life



https://www.scribbr.com/language-rules/best-grammar-checker

Research software development should be a **streamlined process** supported by appropriate tools



Best Practices for Sustainable Software in the Natural and Engineering Sciences aims better research software by providing guidance and tools

- By providing domain-relevant guidelines, tools, infrastructure, and training, the project seeks to support awareness and uptake of best practices during the entire software life cycle.

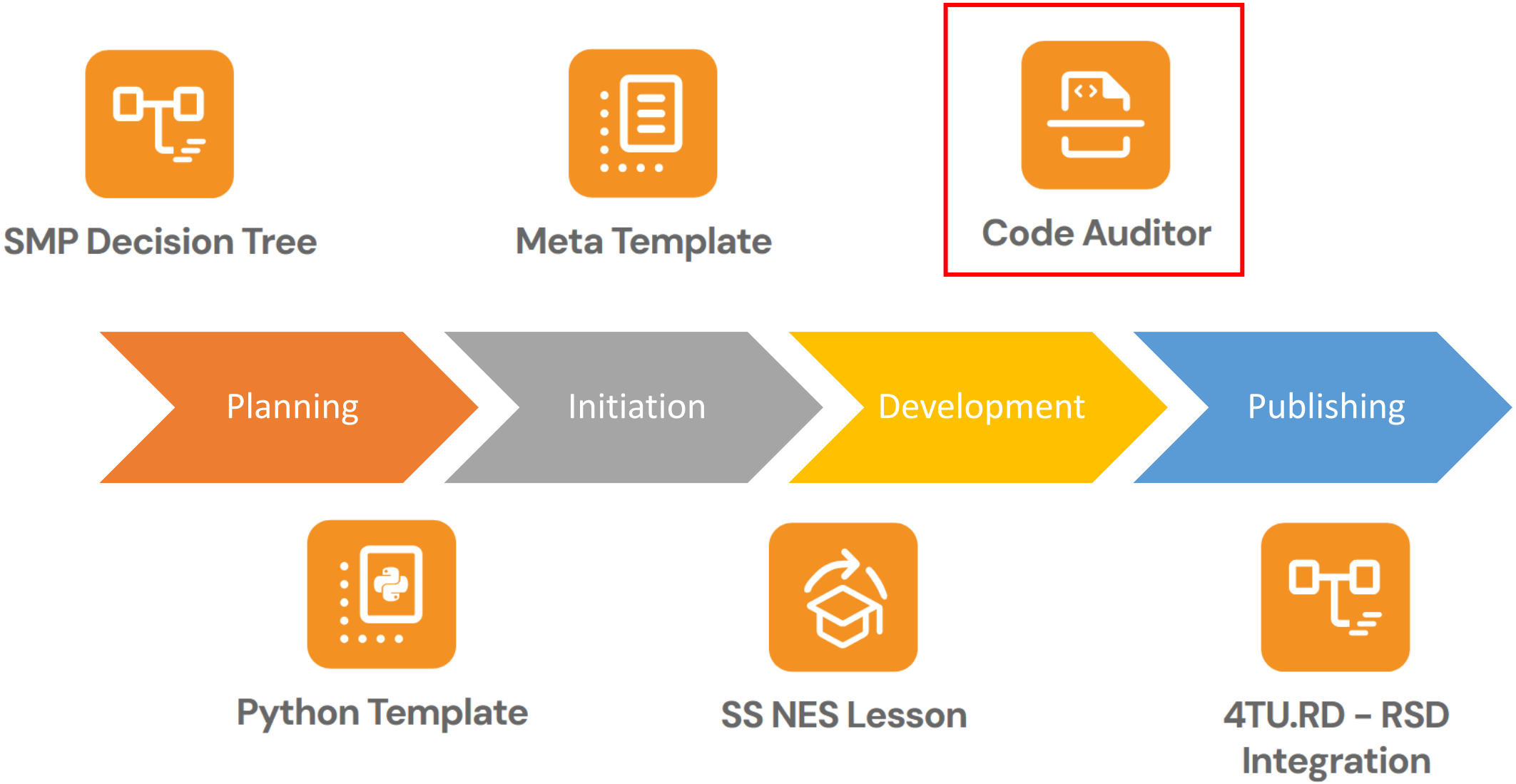
- WP 1: Develop guidance on sustainable software.
  WP 2: Provide tools to facilitate application of the guidance.
  WP 3: Improve and integrate digital infrastructure for sustainable software.
  WP 4: Training, community building, and dissemination.

- Project partners are Netherlands eScience Centre, University of Twente, 4TU.ResearchData, TU Delft, University of Groeningen, KNMI, University of Utrecht, University of Leiden

- The project is funded by the TDCC NES Fund Call 2023

TDCC-NES
Natural & Engineering Sciences

https://tdcc.nl/projects/project-initiatives-nes/tdcc-nes-bottleneck-projects/best-practices-for-sustainable-software/



https://ss-nes.github.io/

code-auditor

A package and command-line tool to audit code quality and conformity with research software development best practices

- Extract software metadata from a software code base.
- Identify issues, such as missing or conflicting practices.
- Provide suggestions to solve the issues.
- Enable comparison with reference software metadata (e.g., SMP)*.
- Enable automated corrections*.
- Output results in various formats.

https://github.com/SS-NES/code-auditor

How to install?

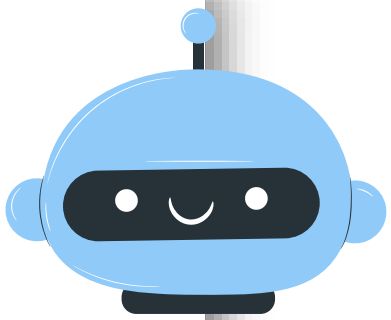
```
Install the latest release
pip install code-auditor

Install most up-to-date version from the repository
pip install git+https://github.com/SS-NES/code-auditor

You can also install it as an isolated tool
pipx install code-auditor
```

How to run on the command line

```
Run a code audit on the local path and print the report to the terminal:

codeauditor <path>


Run a code audit on the code-auditor repository and print the report to the terminal:

codeauditor https://github.com/SS-NES/code-auditor


Run a code audit on the code-auditor repository and save the report as a DOCX file:

codeauditor https://github.com/SS-NES/code-auditor --format docx --output report.docx
```

What is provided at the end of the audit


```
"metadata": {
  "name": [
    {
      "val": "code-auditor",
      "analyser": "citation",
      "path": "CITATION.cff"
    },
    {
      "val": "code-auditor",
      "analyser": "packaging_python",
      "path": "pyproject.toml"
    }
  ],
  "doi": [
  "repository_code": [
  "long_description": [
  "keywords": [
  "license": [
  "version": [
    {
      "val": "0.2.1",
      "analyser": "citation",
      "path": "CITATION.cff"
    }
  ],
  "date_released": [
  "python_dependencies": [
  "readme_file": [
  "version_control": [
  "description": [
  "license_file": [
  "authors": [
  "license_name": [
    {
      "val": "GNU General Public License v3 (GPLv3)",
      "analyser": "packaging_python",
      "path": "pyproject.toml"
    }
```

Code Analysis Report

Analysis report on code quality and conformity to software development best practices for **Code Scholar**. The software is located at `D:\Personal\projects\python\codescholar`.

Notices

- Citation file exists.
- Software code exists.
- Documentation exists.
- License file exists.
- Packaging exists.
- Version control exists.

Issues

- click dependency version is not pinned. (requirements.txt)
- docstring_parser dependency has no version specifier. (requirements.txt)
- gitpython dependency has no version specifier. (requirements.txt)
- jinja2 dependency has no version specifier. (requirements.txt)
- pip-requirements-parser dependency has no version specifier. (requirements.txt)
- pymarkdownlnt dependency has no version specifier. (requirements.txt)
- pypandoc dependency has no version specifier. (requirements.txt)
- pyyaml dependency has no version specifier. (requirements.txt)
- No contributing guidelines.

  Contributing guidelines in a document describing how other may contribute to your software project. The document explains how anyone can engage in activities such as formatting code for submission or submitting patches. You can read more on how to build a contributing guidelines on Mozilla Science Lab.

- No code of conduct.

  A code of conduct is a document that establishes expectations for behavior for your project's participants. Adopting, and enforcing, a code of conduct can help create a positive social atmoshpere for your community. You can read more on code of conduct at Open Source Guides, and use the Contributor Covenant as a template for your own.

- No changelog file.
- No testing.

Illustration by Storyset.com

How to run programmatically

```
# Import package
import codeauditor

# Generate the report
report = codeauditor.audit("https://github.com/SS-NES/code-auditor")

# Get the report output as Markdown
out = report.output(format=codeauditor.report.OutputType.MARKDOWN)

# Display output
print(out)
```

Let's demonstrate!

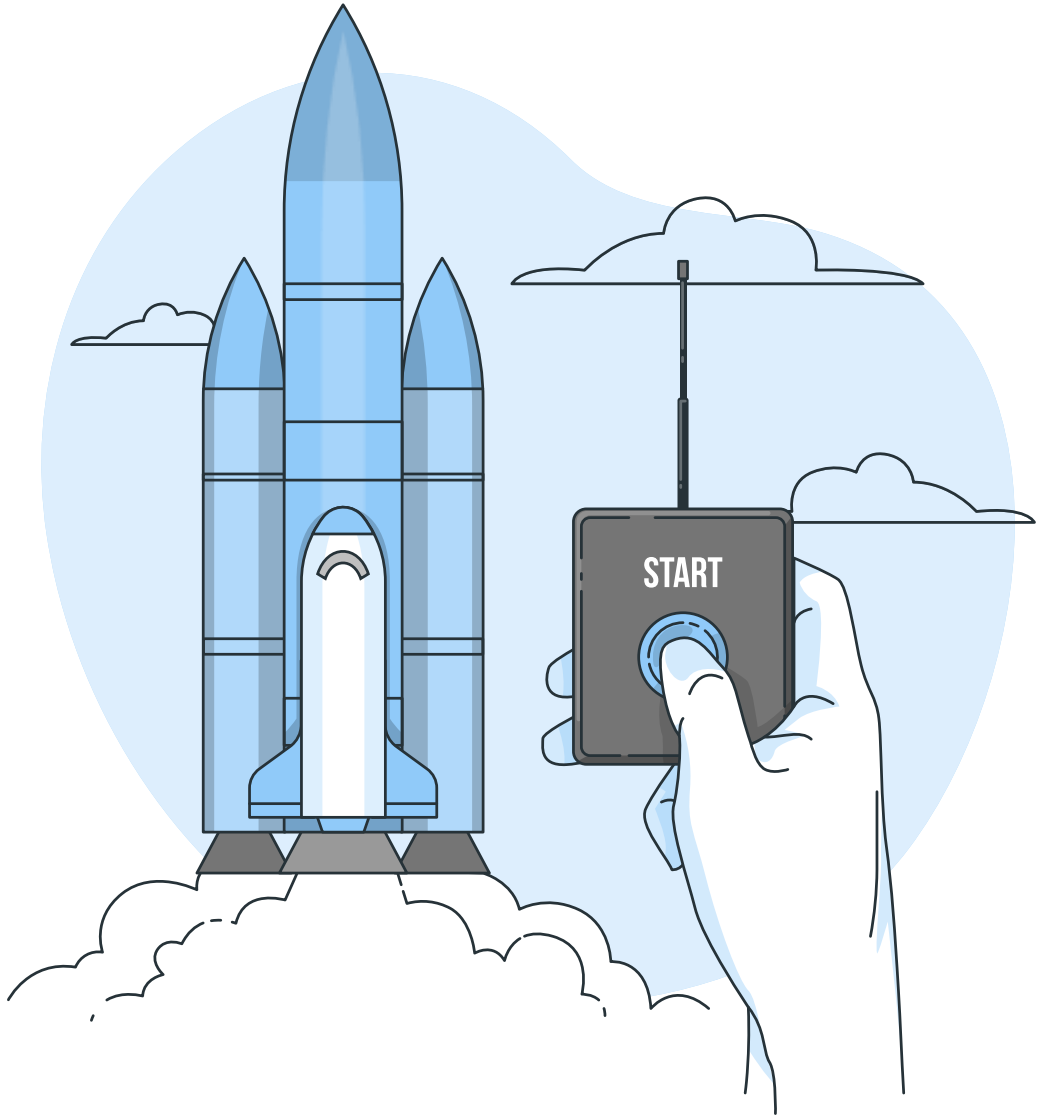


Illustration by Storyset.com

How does **code-auditor** work?

- code-auditor is composed of **analysers** and **aggregators**, which are dedicated to specific aspects of research software such as code quality, license, citation, version control, documentation, packaging, unit testing, etc.
- Analysers analyse **individual files** in a project to extract metadata and identify file (or folder) specific issues.
- For example, **Python code analyser** analyses .py files and identifies if a specific well-know structure is used, if a coding style is consistently in use, if code documentation exists and complete.
- Similarly, a **Git analyser** checks if a local (or remote) repository exists, identifies if best practices are in use, and extracts metadata (e.g., contributors).
- **Aggregators** use the results of related analysers to provide a higher-level summary and identify issues that require information from multiple analysers.
- For example, **code aggregator** uses the results of Python, R, Bash, etc. analysers to provide overall source code completeness.
- Similarly, **version control aggregator** uses the results of Git, SVN, etc. analysers to indicate of version control is in place and its quality.

code-auditor **reports the issues** and **integrates with other tools** to solve them

- Audit results can be obtained as **human- and machine-readable** reports in various formats.
  - HTML, Markdown, reStructuredText, RTF, Word Document for humans.
  - JSON and YAML for machines.
- Audit results include a **summary** of performed audit, identified **best practices**, identified **issues**, **suggestions** for the identifies issues, and extracted **metadata**.
- Audit results can be **compared to reference metadata** to identify deviations from the reference, e.g. **SMP***.
- Audit results can be used as **input to software templating** tools to correct existing information or add additional ones in a structured manner*.
- Audit results can also be used as **input to SMP generation** tools to update SMP according to the current state of the research software project*.

Illustration by Storyset.com

Let's discuss!

Illustration by Storyset.com

fairly toolset facilitates access to research data, and enables local data management and easy publishing

- The toolset includes **fairly** Python library, **fairly-cli** command line tool, and **jupyter-fairly** JupyterLab extension that provide means to create and manage research datasets and publish them at various data repositories in a unified way.
- The toolset reduces the time and effort required to publish open research data, and facilitates better data sharing during the whole research lifecycle.
- The project was funded by the NWO Open Science Fund

https://github.com/ITC-CRIB/fairly

UNIVERSITY OF TWENTE. | ITC

TU Delft

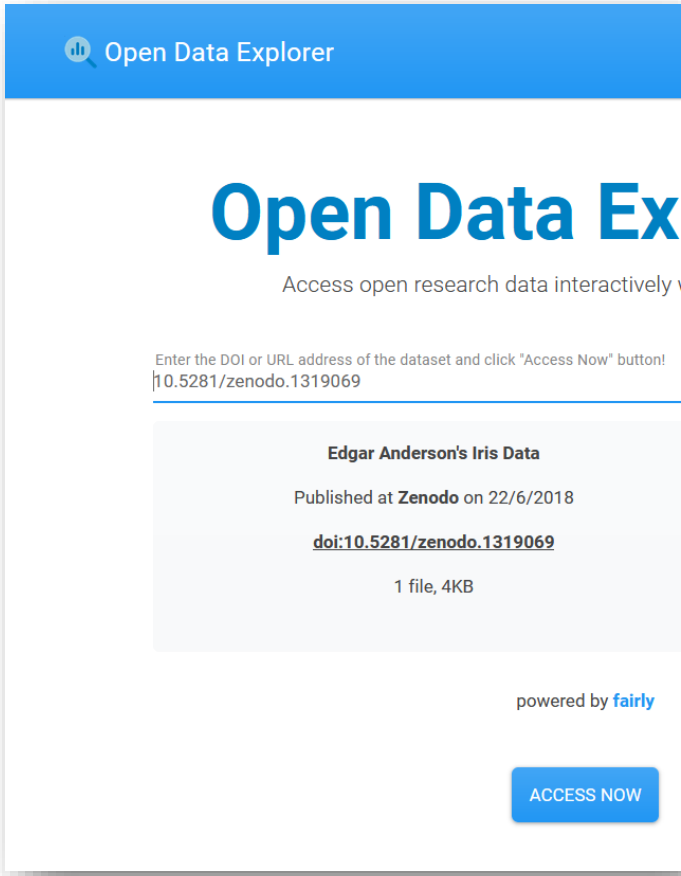
4TU.ResearchData
SCIENCE • ENGINEERING • DESIGN

```python
import fairly

# Create a local dataset
dataset = fairly.create_dataset('/path/datase

# Set metadata
dataset.set_metadata({
    "title": "My wonderful dataset",
    "license": "CC BY 4.0",
    "keywords": ["FAIR", "data"],
    "authors": [
        "0000-0002-0156-185X",
        {
            "name": "John",
            "surname": "Doe",
            "role": "contributor",
        },
    ],
})

# Add data files and folders
dataset.add_files([
    "README.txt",
    "*.csv",
    "train/*.jpg",
    "test/*.jpg"
])

# Upload to the remote data repository
remote_dataset = dataset.upload("4tu")

# Change metadata
dataset.metadata["license"] = "MIT"

# Synchronize the remote dataset with the loc
dataset.synchronize()
```

Open Data Explorer enables direct access to open research data for interactive exploratory data analysis

- The platform provides researchers interactive computing environments where (large) research datasets are readily available without downloading together with example exploratory data analysis notebooks.
- The platform resolves the need for a separate environment to explore research data, reduces the time to explore research data, and reduces the amount of data that is transferred but not effectively utilized.
- The project was funded by the SURF DCC Investment for Digital Infrastructure Call

https://opendataexplorer.org



UNIVERSITY OF TWENTE. | ITC

TU Delft

4TU.ResearchData
SCIENCE • ENGINEERING • DESIGN

Open Data STAC aims to enable make geospatial open research data findable and accessible

- A large amount of open research data is **spatiotemporal** by its nature.
- However, due to the limitations of the existing data repositories, spatiotemporal information stays **mostly hidden**, and researchers are limited to use basic textual metadata and simple search functionalities to find geospatial data, which is **mostly unsuccessful**.
- **OpenSTAC** aims to extract useful spatiotemporal information from open research datasets and make it accessible through an online catalog that can be accessed by a large ecosystem of tools and applications based on STAC.
- The project is funded by the NWO Open Science Fund Call 2023

The platform will be publicly available at the end of 2025

STAC

CLOUD-NES aims to stimulate the use of cloud-native methods to publish, access, and process research data

- Cloud-based data processing is ramping up as a modern digital competence.
  However, downloading data and analysing it locally is still a standard practice for most researchers.
- Sometimes this is because the data is not provided in a format that well suits cloud-based processing.
  But it is also common that the skills necessary for cloud-based data access and processing are lacking.
- CLOUD-NES aims to stimulate the use of cloud-native methods by building a prototype cloud-native data repository, transforming selected datasets in traditional formats into cloud-native ones, demonstrating the benefits of cloud-native approaches with reliable and reproducible benchmarks, developing open training material and providing tailored training to the researchers and data providers to improve their skills in using cloud-native methods.
- The project is funded by the TDCC-NES Challenge Projects Call 2024

UNIVERSITY OF TWENTE. | ITC

netherlands eScience center

SURF

KNMI

Started in October!

We will be happy to be in **contact**!

Dr. Ing. Serkan Girgin MSc

Head of Department
Center of Expertise in Big Geodata Science

Associate Professor
Department of Geo-information Processing

Faculty ITC, University of Twente

s.girgin@utwente.nl
https://linkedin.com/in/serkan-girgin/

Faculty of Geo-information Science and Earth Observation (ITC)
Centre of Expertise in Big Geodata Science (CRIB)

UNIVERSITY OF TWENTE. | ITC

https://itc.nl